Informe del Trabajo Práctico
Procesamiento de Imágenes I

Integrantes:

Gómez, Mercedes G-6061/5
Malvestiti, Carolina Anna
Nebbia, María Sofía N-1301/3

Fecha de entrega: 20/04/26

1° Problema: Ecualización Local del Histograma

El objetivo fue implementar una función que aplique la ecualización local del histograma sobre una imagen en escala de grises. A diferencia de la ecualización global, que considera todos los píxeles de la imagen para construir el histograma, esta técnica trabaja con un vecindario de tamaño M×N centrado en cada píxel, calculando una transformación de intensidad adaptada a las condiciones locales de cada zona.

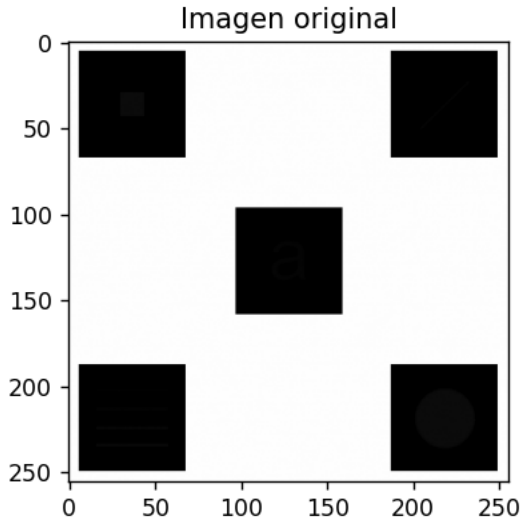
La función implementada, ecualizacion_local_histograma, recibe como parámetros la imagen original y las dimensiones del vecindario (M y N). Para cada píxel de la imagen, se extrae la región local correspondiente y se calcula su histograma de niveles de grises a traves de np.histogram, obteniendo las frecuencias absolutas de cada valor de intensidad entre 0 y 255.

Este histograma se normaliza dividiendo por la cantidad total de píxeles del vecindario (M×N), convirtiéndo en una distribución de probabilidad. A partir de esta distribución normalizada se calcula la suma acumulada, que actúa como función de transformación. El nuevo valor del píxel central se obtiene multiplicando el valor acumulado correspondiente a su intensidad original por 255, escalando al rango [0, 255].

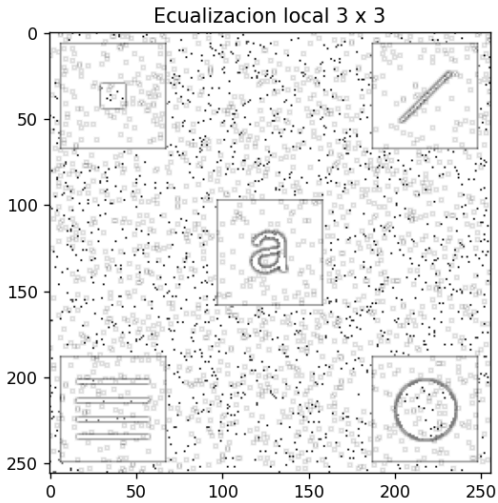
```python
def ecualizacion_local_histograma(imagen, M, N):
    filas, columnas = imagen.shape
    margen_y = M // 2
    margen_x = N // 2

    imagen_extendida = cv2.copyMakeBorder(imagen, margen_y, margen_y, margen_x, margen_x, cv2.BORDER_REPLICATE)

    imagen_resultado = np.zeros_like(imagen, dtype = np.uint8)

    for i in range(filas):
        for j in range(columnas):

            region = imagen_extendida[i:i+M, j:j+N]

            histograma, _ = np.histogram(region, bins=256, range=(0, 256))

            histograma = histograma / (M * N)

            suma_acumulada = np.cumsum(histograma)

            pixel_central = imagen_extendida[i + margen_y, j + margen_x]
            nuevo_valor = 255 * suma_acumulada[pixel_central]

            imagen_resultado[i, j] = np.uint8(nuevo_valor)

    return imagen_resultado
```

Un aspecto importante que resolvimos fue el tratamiento de los píxeles ubicados en los bordes de la imagen. Al centrar la ventana sobre un píxel cercano al borde, parte del vecindario queda fuera del área válida. Para solucionar esto se utilizó cv2.copyMakeBorder con BORDER_REPLICATE, que extiende la imagen replicando los valores de los bordes. De esta forma, todas las ventanas tienen el tamaño correcto y la ecualización puede aplicarse de manera uniforme sobre toda la imagen, incluyendo las esquinas.
Se aplicó la función sobre la imagen provista en el problema (Imagen_con_detalles_escondidos.tif) con distintos tamaños de ventana para analizar su influencia en los resultados.

ecualización original

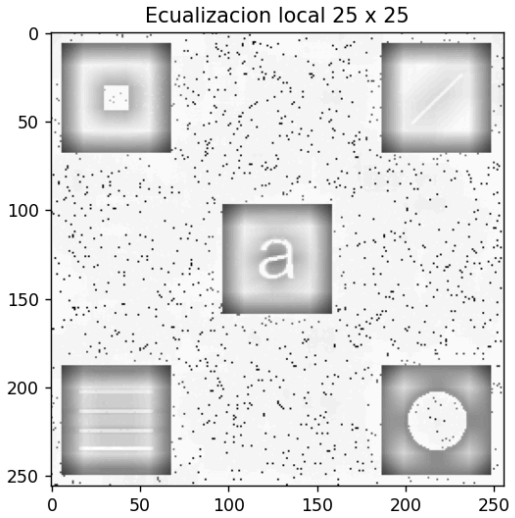


ecualización local 3x3



Con una ventana de 3×3 la ecualización es muy sensible a variaciones locales finas: los detalles se revelan con claridad, pero el resultado presenta mayor ruido y artefactos, ya que el histograma se construye con muy pocos píxeles y resulta estadísticamente poco representativo.

ecualización local 25×25



Al aumentar el tamaño de la ventana, la transformación se vuelve más suave y los detalles son menos ruidosos, aunque también comienzan a perder definición en las zonas de muy bajo contraste local. Con ventanas grandes, el comportamiento se aproxima al de la ecualización global, donde se pierde la localidad del análisis y los detalles sutiles dejan de ser distinguibles.

Conclusión:

La ecualización local mejora bastante la visibilidad de detalles en regiones donde el contraste local es bajo, adaptando la transformación a las condiciones de cada zona de la imagen. El padding replicado garantiza una correcta aplicación en los bordes. El tamaño de ventana influye directamente en el resultado: ventanas pequeñas (3×3) revelan detalles finos con mayor sensibilidad pero más ruido, mientras que ventanas más grandes producen resultados más suaves pero menos reactivos a variaciones locales. Luego de aplicar la ecualización local a la imagen original, fue posible identificar distintas figuras ocultas dentro de las regiones oscuras: un cuadrado, un círculo, la letra "a" y líneas rectas, las cuales no eran visibles ni en la imagen original ni con la ecualización global.

2° Problema: Corrección de Múltiple Choice

El objetivo de este problema fue desarrollar un algoritmo capaz de corregir automáticamente exámenes de opción múltiple a partir de imágenes digitales. Cada examen contiene un encabezado con tres campos (Name, Date, Class) y 10 preguntas con cuatro opciones posibles (A, B, C, D) organizadas en una grilla. El algoritmo debe detectar la estructura del formulario, identificar las respuestas escritas, validar los campos del encabezado y generar una imagen resumen con los resultados.

Preprocesamiento

En primer lugar, la imagen se carga en escala de grises y se binariza mediante cv2.adaptiveThreshold con umbralización adaptativa gaussiana. Se utiliza THRESH_BINARY_INV para obtener el contenido escrito en blanco sobre fondo negro, facilitando las operaciones morfológicas y el análisis de componentes conectadas.

La umbralización adaptativa gaussiana compara cada píxel con el promedio ponderado de su vecindario de 15×15 píxeles, restándole 8 al umbral calculado para ajustar la sensibilidad. Esta técnica resulta más robusta que un umbral fijo global porque se adapta a las variaciones locales de iluminación presentes en la imagen, permitiendo binarizar correctamente tanto zonas claras como oscuras del formulario.

Sin embargo, para la detección de los subrayados del encabezado (Name, Date y Class), la binarización adaptativa no resultó efectiva debido al bajo contraste local de esas líneas finas. Por este motivo, la detección de los campos del encabezado se realiza directamente sobre la imagen en escala de grises utilizando un umbral fijo de intensidad (valor menor a 150), lo que permite identificar correctamente las zonas oscuras de los subrayados. De esta forma el algoritmo utiliza dos estrategias de umbralización complementarias: adaptativa para el contenido de las celdas de preguntas y fija para la estructura del encabezado.

```
def cargar_y_binarizar(path):
    img = cv2.imread(path, cv2.IMREAD_GRAYSCALE)
    bin_inv = cv2.adaptiveThreshold(img, 255, cv2.ADAPTIVE_THRESH_GAUSSIAN_C, cv2.THRESH_BINARY_INV, 15, 8)

    return img, bin_inv
```

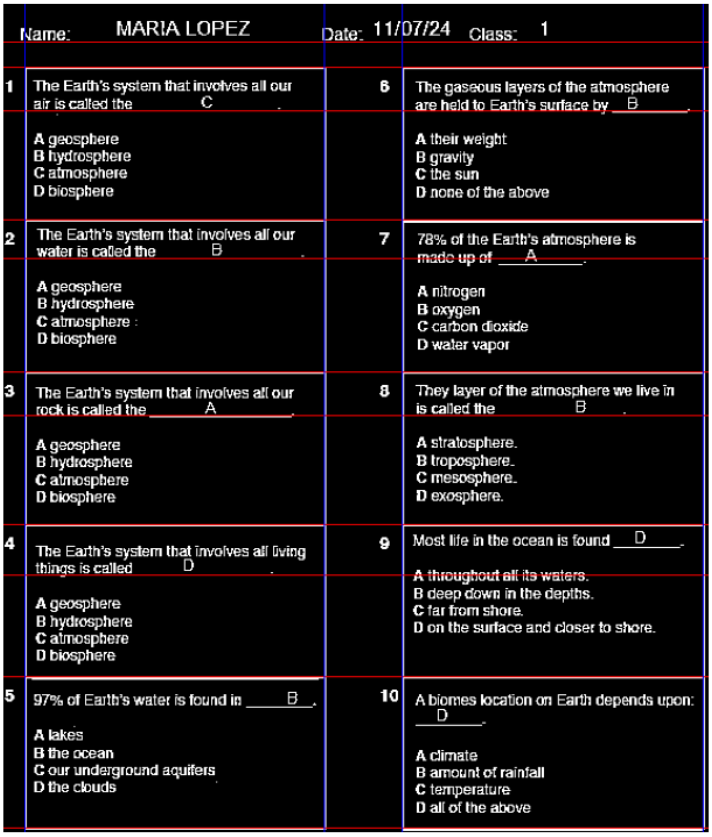
Detección de la grilla

Para detectar las líneas de la grilla se construyeron dos kernels rectangulares usando np.ones: uno horizontal de 1 fila y ancho equivalente al 10% del ancho de la imagen, y uno vertical de alto equivalente al 10% del alto y 1 columna. Sobre cada kernel se aplicó erosión y luego dilatación (cv2.erode y cv2.dilate). La erosión elimina todo elemento más corto que el kernel, y la dilatación restaura el tamaño original de lo que sobrevivió. El resultado son dos imágenes: una con solo las líneas horizontales largas y otra con solo las líneas verticales.

```python
def detectar_lineas(bin_inv):

    alto, ancho = bin_inv.shape
    kernel_h = np.ones((1, ancho // 10), dtype=np.uint8)

    kernel_v = np.ones((alto // 10, 1), dtype=np.uint8)

    lineas_h = cv2.erode(bin_inv, kernel_h)
    lineas_h = cv2.dilate(lineas_h, kernel_h)

    lineas_v = cv2.erode(bin_inv, kernel_v)
    lineas_v = cv2.dilate(lineas_v, kernel_v)

    return lineas_h, lineas_v
```

ACLARACIÓN: Durante el desarrollo implementamos una función auxiliar de visualización (ver_grilla) que permitió verificar que las líneas detectadas coincidieran correctamente con la estructura del formulario. La imagen muestra el resultado de superponer las coordenadas detectadas sobre la imagen binarizada: las líneas horizontales marcadas en rojo y las verticales en azul. Esta función fue utilizada únicamente como herramienta de debug para verificar nuestro algoritmo y no forma parte del procesamiento final.

grilla detectada



Validación del encabezado

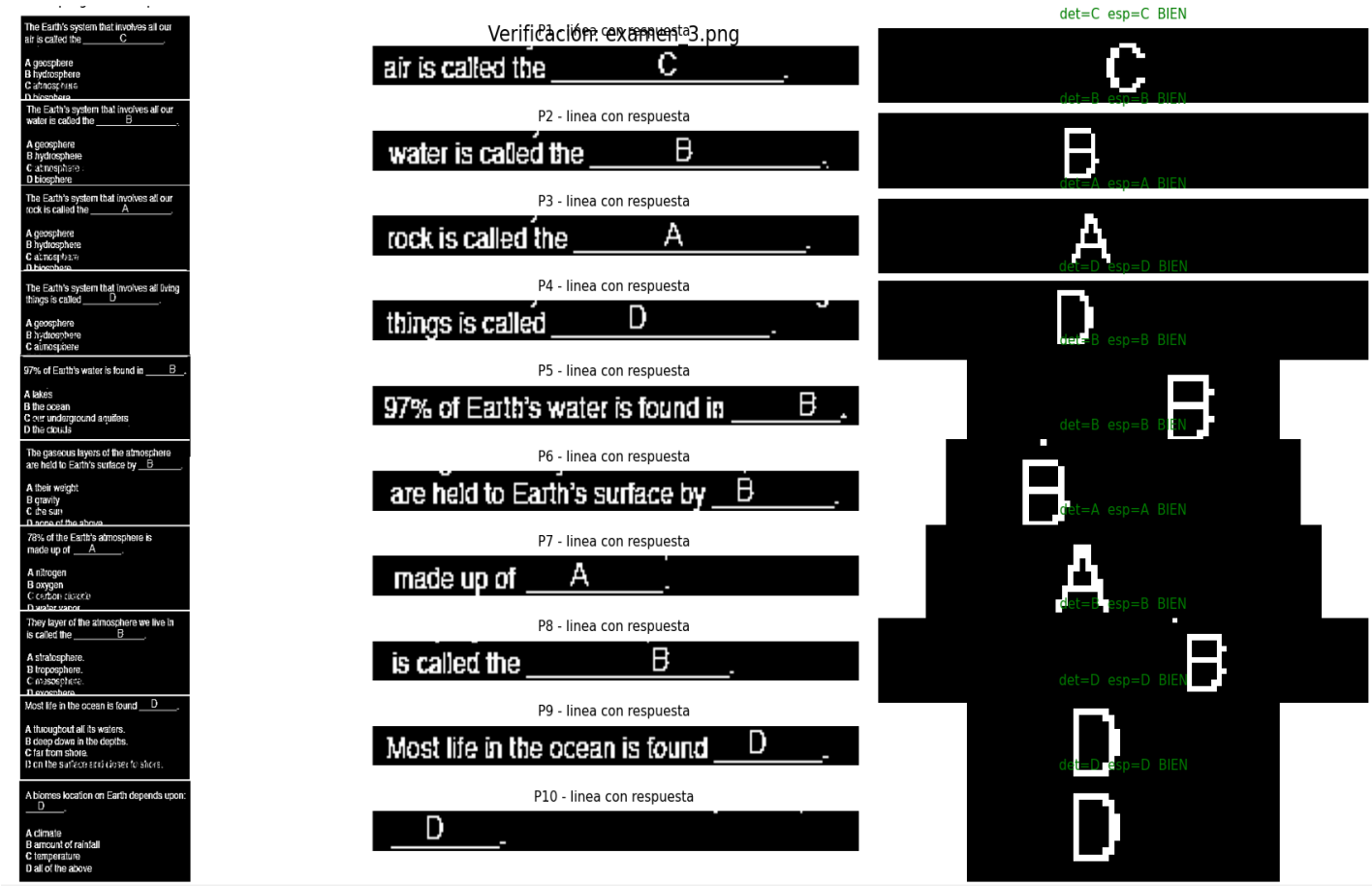
Para detectar los campos del encabezado se implementó la función detectar_campos_encabezado, que encuentra los subrayados de Name, Date y Class directamente sobre la imagen en escala de grises. El método recorre la fila correspondiente al primer subrayado detectado por la grilla y busca rachas de píxeles oscuros (valor menor a 150) de al menos 30 píxeles consecutivos. Cada racha encontrada corresponde a uno de los tres subrayados, y a partir de sus coordenadas horizontales se recorta la zona de texto ubicada aproximadamente 20 píxeles por encima.

La validación de cada campo se realiza contando las componentes conectadas dentro del recorte binarizado. Para Name se verifica que haya entre 3 y 25 componentes y que exista un espacio significativo entre dos palabras. Para Date se verifica que haya exactamente 8 componentes formando una sola palabra, que corresponde al formato dd/mm/aa. Para Class se verifica que haya exactamente 1 componente, es decir un único carácter.

```python
def validar_name(celda_bin):
    n, cajas = contar_componentes(celda_bin, area_min=15)
    if n < 3:
        return 'MAL'
    if n > 25:
        return 'MAL'
    if hay_espacio_entre_palabras(cajas, factor=2.0):
        return 'OK'
    return 'MAL'

def validar_date(celda_bin):
    n, cajas = contar_componentes(celda_bin, area_min=15)

    if n != 8:
        return 'MAL'

    if hay_espacio_entre_palabras(cajas, factor=2.0):
        return 'MAL'

    return 'OK'

def validar_class(celda_bin):
    n, _ = contar_componentes(celda_bin, area_min=15)
    return 'OK' if n == 1 else 'MAL'
```

Extracción de respuestas

Para cada una de las 10 preguntas, se recorta la celda correspondiente y dentro de ella se busca la línea de completado: la primera fila que contenga una racha de al menos 30 píxeles blancos consecutivos. Una vez localizada esa línea, se recorta la franja inmediatamente superior (aproximadamente 15 píxeles de alto), donde el alumno escribe la letra de respuesta. Luego se verifica si hay más de una letra en esa franja: si se detectan dos o más componentes con altura ≥ 6 px y área ≥ 20 px, la respuesta se clasifica como INVALIDA.



ACLARACION: Durante el desarrollo se utilizó una función auxiliar de visualización (verificar_recortes) que permitió inspeccionar la extracción de respuestas en tres etapas: la celda completa con el enunciado, la franja recortada que contiene la línea de respuesta, y la letra identificada con la comparación contra la respuesta correcta. Esta herramienta fue clave para ajustar los parámetros del algoritmo y verificar que los recortes fueran correctos. Al igual que ver_grilla, no forma parte del procesamiento final sino que fue utilizada exclusivamente durante el desarrollo.

Identificación de letras

Para distinguir entre A, B, C y D se analiza morfológicamente la componente más grande detectada en la franja de respuesta. El criterio principal es el conteo de contornos internos (huecos) usando cv2.findContours con RETR_CCOMP:

C: 0 huecos internos (trazo abierto).
B: 2 o más huecos (dos semicírculos cerrados).
A o D: exactamente 1 hueco. Para distinguirlas se analiza la proyección vertical de píxeles blancos: si el pico en las primeras 3 columnas es al menos el doble del promedio total se clasifica como D (palo vertical a la izquierda); de lo contrario, como A.

```
def identificar_letra(zona_bin):
```

Generación de imagen de resultados

Una vez procesados todos los exámenes, se genera una imagen resumen que muestra el recorte del campo Name de cada alumno con un borde de color: verde para los aprobados (≥ 6 correctas) y rojo para los desaprobados.

```
for idx, (nombre, crop, puntaje, aprobado) in enumerate(resultados):
    y = margen + idx * (alto_celda + margen)
    x = margen

    color = (0, 180, 0) if aprobado else (0, 0, 200)

    crop_color = cv2.cvtColor(crop, cv2.COLOR_GRAY2BGR)

    crop_resized = cv2.resize(crop_color, (ancho_celda, alto_celda))
    imagen[y:y+alto_celda, x:x+ancho_celda] = crop_resized
    grosor = 4
    cv2.rectangle(imagen, (x - grosor, y - grosor),
                  (x + ancho_celda + grosor, y + alto_celda + grosor), color, grosor)

return imagen
```

Dificultades encontradas

Uno de los desafíos principales fue la identificación de las letras. La clasificación basada en contornos internos y proyección vertical resultó efectiva y relativamente estable frente a esa variabilidad, aunque necesitamos ajustar el umbral de área mínima para filtrar ruido sin eliminar partes válidas de las letras.

Otro punto crítico fue la detección de respuestas con múltiples opciones marcadas. Implementamos un filtrado de componentes por tamaño mínimo (altura y área) para evitar que manchas de ruido sean interpretados como letras adicionales, sin que eso afecte la detección de casos realmente inválidos.

La validación del campo Name también presentó dificultades para distinguir si los trazos separados pertenecían a dos palabras distintas o simplemente a caracteres con separación natural entre sí. Esto se resolvió exigiendo que la separación máxima sea significativamente mayor al promedio y que existan al menos 3 componentes a cada lado.

Otro desafío fue la detección de los campos del encabezado. La binarización adaptativa no resultó efectiva para identificar los subrayados de Name, Date y Class debido al bajo contraste local de esas líneas finas. Esto llevó a implementar una estrategia distinta, trabajando directamente sobre la imagen en escala de grises con un umbral fijo de intensidad para detectar las rachas oscuras de los subrayados.

Conclusión:

El algoritmo logró detectar automáticamente la estructura del formulario, extraer y clasificar las respuestas, y validar los datos del encabezado sin utilizar coordenadas fijas como entrada. La estrategia morfológica para la detección de la grilla resultó robusta frente a variaciones entre exámenes. La identificación de letras basada en contornos internos y proyección vertical permitió distinguir correctamente entre A, B, C y D en la mayoría de los casos. La imagen de salida generada ofrece una visualización clara e inmediata del resultado de todos los exámenes evaluados.